Policy at the Actuation Boundary: Hardware-Rooted Safety Enforcement for AI-Controlled Robots

Joel Nishanth Ponukumatla
Offlyn Verify Core Project
joelnishanthreddy@gmail.com · hi@offlyn.ai

Draft v0.2 – June 17, 2026

Abstract

Large foundation models and agentic planners are increasingly being connected to robots, drones, industrial manipulators, and other cyber-physical systems. This creates a new safety problem: a model may propose an unsafe action, a compromised software stack may bypass a software guardrail, or a benign task may become physically unsafe when translated into concrete actuation parameters. Existing safeguards for LLM-enabled robots include prompt-level constraints, pre-execution validators, temporal-logic planners, runtime assurance frameworks, safety filters, and ROS-level middleware. These approaches are valuable, but many remain inside the same software trust domain as the planner, executor, operating system, or middleware they are intended to constrain.

This paper proposes Offlyn Verify Core, a hardware-rooted policy gate placed at the actuation boundary between the autonomy stack and the physical command path. The key idea is to move final action authorization into a small, deterministic, separately controlled enforcement point that releases actuator-bound commands only after checking signed policy capsules, actor identity, command schema, physical bounds, freshness, and selected state predicates. We define the Actuation Boundary Enforcement Point abstraction, an action authorization interface, fail-closed enforcement semantics, and an evidence model for allow and deny decisions. The paper positions Offlyn Verify Core relative to recent work on LLM robot guardrails, pre-execution safety gates, zero-trust policy models for agentic cyber-physical systems, runtime assurance, control barrier functions, and mixed-criticality robotics. The contribution is not a claim that hardware alone makes robots safe. Rather, it is a concrete systems architecture for reducing the trusted computing base of AI-controlled robotic actuation by making the final authorization boundary smaller, auditable, tamper-resistant, and physically harder to bypass. The current artifact is a reproducible software prototype that validates the architecture, interfaces, and defense properties; hardware implementation is a staged future path from TPM-backed signing to microcontroller, FPGA, and ASIC enforcement.

1 Introduction

Robotics is entering a period in which high-level language models and agentic planners are being given increasingly direct paths to physical action. A user may issue a natural-language task, a foundation model may decompose that task into tool calls, and a robotics middleware layer may translate those tool calls into movement, grasping, navigation, or manipulation commands. The benefit is flexibility: robots can be commanded in ordinary language and adapted to new environments with less hand-coded task logic. The risk is that the final output of a probabilistic planner becomes a physical event.

The safety problem changes when an AI system can move mass, apply force, enter a restricted zone, operate around humans, or modify industrial process state. A chatbot error may be reversible; a robot action may not be. For this reason, recent work has proposed new safety guardrails for LLM-enabled robots [1], neurosymbolic pre-execution safety gates and task safety contracts [2], ROS-level executive layers with action validators and audit logs [3], and zero-trust policy primitives for agentic cyber-physical systems [4]. Runtime assurance has also long studied how an unverified advanced controller can be monitored or overridden by a trusted safety controller [6, 7]. Control barrier functions and shielding provide mathematically

grounded methods for keeping control inputs within safe sets [8, 9]. Mixed-criticality robotics work shows that hardware-enforced isolation and safety I/O cells can reduce interference between real-time control and best-effort software [10].

These lines of work are complementary, but they leave an architectural gap for AI-controlled robots: where exactly should final authority over physical action live? If the policy engine, guardrail, action validator, and audit log all execute inside the same robot computer, ROS graph, operating system, container runtime, or agent framework as the planner, then a single compromised or misconfigured software trust domain may be able to alter, bypass, or disable the constraint. This paper argues that the final policy enforcement point should be moved as close as practical to the physical actuation path.

We call this boundary the Actuation Boundary Enforcement Point. It is the last mandatory authorization surface before a command can affect robot motion, force, position, tool activation, or other safety-relevant physical state. The proposed Offlyn Verify Core design treats this boundary as a small hardware-rooted component that accepts canonical action requests and releases actuator-bound commands only when deterministic checks pass. It is not a replacement for classical functional safety, runtime assurance, CBFs, formal planning, or human supervision. It is an additional, narrower trust boundary that makes it harder for an AI planner or compromised host to convert unsafe intent into physical motion.

1.1 Research question

The central research question is:

> Can a small hardware-rooted actuation policy gate reduce the trusted computing base for AI-controlled robots while preserving useful real-time actuation performance and auditability?

This framing deliberately avoids two overclaims. First, it does not claim that policy enforcement is new. Second, it does not claim that hardware enforcement is sufficient for safety. The claim is narrower: for AI-controlled robots, there is value in relocating final authorization from a large, modifiable software stack to a small, deterministic, hardware-rooted boundary.

1.2 Contributions

This paper makes four contributions:

1. It defines the Actuation Boundary Enforcement Point abstraction for AI-controlled robots: a mandatory, fail-closed authorization point between AI-generated action requests and physical actuation.
2. It proposes the Offlyn Verify Core architecture, including signed policy capsules, canonical action requests, freshness checks, actor identity, bounded state predicates, command release semantics, and tamper-evident evidence records.
3. It distinguishes Offlyn Verify Core from adjacent software guardrails, pre-execution validators, runtime assurance architectures, CBF-based safety filters, and mixed-criticality hardware isolation.
4. It provides a reproducible open-source artifact demonstrating denial of unsafe commands, replay attempts, rollback attempts, unsigned policy updates, and direct actuator bypass, together with an evaluation protocol for measuring safety leakage, false rejection, latency overhead, bypass resistance, replay resistance, and audit completeness.

2 Prior Art and Gap

Recent research has already identified important pieces of the problem. RoboGuard addresses safety risks in LLM-enabled robots by grounding safety rules into context-dependent temporal logic constraints and synthesizing plans that satisfy those constraints [1]. SafeGate introduces a neurosymbolic gate for LLM-controlled robots and task safety contracts that decompose accepted commands into invariants, guards, and abort conditions [2]. ROSClaw provides a model-agnostic ROS 2 executive layer with dynamic capability

Table 1: Positioning Offlyn Verify Core against adjacent approaches. A check mark indicates the primary focus of the cited work, not an exhaustive capability claim.

Approach	Main focus	LLM robot safety	Formal policy or verification	Hardware-rooted boundary	Direct actuation placement
RoboGuard [1]	Contextual safety rules and temporal-logic plan synthesis for LLM-enabled robots	Yes	Yes	No	No
SafeGate [2]	Neurosymbolic pre-execution gate and task safety contracts	Yes	Yes	No	No
ROSClaw [3]	ROS 2 executive layer with validation and audit logging	Yes	Partial	No	Middleware
ZTPM [4]	Zero-trust policy primitives and physical impact tiers	Yes	Policy model	No	Policy boundary
EHV [5]	Hardware-rooted runtime enforcement for general agentic AI systems	General agents	Yes	Yes	Not robot-specific
Simplex/RTA [6, 7]	Runtime override of unverified controllers	Not LLM-specific	Yes	Varies	Controller authority
CBF/shielding [8, 9]	Mathematical safety filters over states and controls	Not LLM-specific	Yes	No	Control input filter
Jiao [10]	Mixed-criticality partitioning and safety I/O cells	Not LLM-specific	Safety communication	Yes	Control platform I/O
Offlyn Verify Core	Hardware-rooted authorization for AI-originated robotic action requests	Yes	Deterministic policy gate	Simulated / hardware path	Actuation boundary

discovery, observation normalization, pre-execution validation, and structured audit logging [3]. ZTPM proposes typed zero-trust policy primitives and physical impact tiers for agentic cyber-physical systems [4]. EHV explores a hardware-rooted zero-trust enforcement architecture for agentic AI systems more generally, using trusted execution environments, policy synchronization, and machine-readable audit logging [5].

Classical runtime assurance and Simplex-style architectures address the problem of using unverified high-performance controllers in safety-critical systems by monitoring and, when necessary, switching to a trusted controller [6, 7]. Safety filters based on control barrier functions can modify or reject control actions to maintain forward invariance of a safe set [8]. Shielding approaches in reinforcement learning similarly block unsafe actions at runtime based on formal models or abstractions [9]. Jiao addresses mixed-criticality robotics by combining static partitioning, hardware-level override capability, and safety communication layers to preserve isolation in consolidated robot platforms [10].

The gap is not absence of safety checking. The gap is the trust placement of the final action authority. Table 1 summarizes the distinction along dimensions most relevant to the actuation-boundary question.

Table 2 provides a finer-grained comparison along the specific defense dimensions that Offlyn Verify Core targets.

Offlyn Verify Core is not intended to replace runtime assurance, formal verification, control-theoretic safety filters, or robot middleware guardrails. These systems address complementary layers of the safety stack. Rather, Offlyn Verify Core narrows the final command-release boundary by placing a small, deterministic authorization point between untrusted AI-generated action proposals and actuator execution. The contribution is the combination of: (i) AI-originated action requests, (ii) a canonical action authorization interface, (iii) signed policy capsules with rollback protection, (iv) hardware-rooted fail-closed release of actuator-bound commands with replay defense, and (v) evidence records for each allow or deny outcome, backed by an open reproducible artifact. To the best of our current literature scan, this exact systems framing is not yet established as a peer-reviewed robotics paper. The closest research neighbors are ZTPM, SafeGate, ROSClaw, EHV, Simplex/RTA, and Jiao; the proposed design should therefore be presented as an integration and boundary-placement contribution rather than as a claim that policy enforcement or robot

Table 2: Feature-level comparison of related approaches on defense dimensions targeted by Offlyn Verify Core. "–" indicates the dimension is not a stated focus of the cited work.

Approach	Robot-focused	Enforcement placement	Hardware-rooted boundary	Signed policy capsules	Replay defense	Rollback defense	Open artifact
RoboGuard	Yes	Planner level	No	–	–	–	No
SafeGate	Yes	Pre-execution	No	–	–	–	No
ROSClaw	Yes	ROS middleware	No	–	–	–	Partial
EHV	General	TEE runtime	Yes	Yes	–	–	No
RTA / Simplex	Varies	Controller switch	Varies	–	–	–	Varies
Offlyn Verify Core	Yes	Actuation boundary	Simulated / hardware path	Yes	Yes	Yes	Yes

safety monitoring is itself novel.

3 System Model

3.1 Robot stack

We model an AI-controlled robot as a pipeline with five logical layers:

1. **User or task source**: natural-language command, fleet scheduler, operator console, or autonomous mission planner.
2. **AI planner**: LLM, vision-language-action model, hierarchical planner, or multi-agent system that proposes actions.
3. **Executive layer**: middleware that converts proposed actions into structured robot commands, such as ROS 2 service calls, motion goals, end-effector operations, navigation goals, or tool activations.
4. **Actuation boundary**: the last mandatory authorization point before commands reach motor controllers, robot drivers, servo buses, tool controllers, power relays, or safety-relevant I/O.
5. **Physical plant**: robot body, actuators, tools, environment, and nearby humans.

Offlyn Verify Core is placed at layer 4. The executive layer remains useful: it can validate task plans, normalize observations, run simulation checks, call CBF-based filters, and maintain high-level logs. Offlyn Verify Core does not replace those controls. It assumes that the executive layer may be wrong or compromised and therefore does not treat software approval as sufficient for physical action.

3.2 Threat model

The threat model includes the following failures and attacks:

- **Unsafe model output**: the planner proposes a task that is semantically unsafe, physically impossible, or outside the robot's allowed work envelope.
- **Unsafe parameterization**: the high-level task is acceptable, but the generated speed, force, pose, path, torque, or tool parameter is outside policy.
- **Prompt or instruction injection**: the planner is induced to ignore a safety rule or call a privileged action.

- **Middleware compromise**: the host OS, ROS graph, container runtime, action broker, or policy service is compromised.
- **Replay or rollback**: a previously authorized command or older permissive policy is replayed.
- **Audit tampering**: a host attempts to hide denied actions or fabricate compliance evidence.

The proposed design does not solve every physical attack. In particular, it does not prevent mechanical tampering, malicious rewiring around the gate, sensor spoofing outside the measured trust boundary, incorrect policies, flawed hazard analysis, or unsafe hardware design. These are limitations, not edge cases. The architecture is meant to reduce the trusted computing base for action authorization, not to replace safety engineering.

3.3 Security assumptions

Offlyn Verify Core assumes:

1. The actuation path is wired so that safety-relevant commands must pass through the gate or the robot fails closed.
2. The gate has a hardware root of trust or equivalent protected boot path for firmware and policy verification.
3. Policy capsules are signed by an authorized policy authority and include version, epoch, validity interval, and rollback protection.
4. The gate can identify actors, commands, and targets using cryptographic tokens, bus-level authentication, or a deployment-specific equivalent.
5. The gate has access to a bounded set of trusted state predicates required by policy, such as workspace zone, emergency-stop state, human-presence flag, joint limit state, tool armed state, or velocity envelope.

These assumptions should be tested in any implementation. In particular, claim strength depends on whether the gate actually mediates all relevant physical command paths.

4 The Actuation Boundary Enforcement Point

The Actuation Boundary Enforcement Point is a deployment-specific hardware or hardware-rooted component that mediates the conversion of an approved action request into a physical command. It may be implemented as an inline microcontroller, FPGA, safety I/O module, bus gateway, secure coprocessor, trusted motor-controller adapter, or safety PLC extension. The design point is not a single board. The design point is a mandatory enforcement semantics:

> No safety-relevant actuation command is released unless a small, deterministic, independently protected gate authorizes it under a signed policy capsule and records the decision.

4.1 Canonical action request

AI planners produce diverse outputs: text, code, JSON, ROS messages, motion plans, or tool calls. Offlyn Verify Core requires the executive layer to reduce each safety-relevant proposed action into a canonical action request. An example request is shown below.

Listing 1: Example canonical action request.

```
{
  "actor_id": "planner_ur3e_cell_04",
  "request_id": "01jz9c0k3ypb8f6g7z",
  "policy_epoch": 17,
```

```
  "action_type": "move_joint",
  "target_id": "ur3e_joint_3",
  "physical_impact_tier": "T2",
  "parameters": {
    "angle_deg": 42.0,
    "velocity_deg_s": 18.0,
    "max_force_n": 25.0
  },
  "context": {
    "workspace_zone": "cell_a",
    "human_presence": false,
    "task_id": "assembly_204"
  },
  "nonce": "9b21f1a4",
  "signature": "..."
}
```

The request is intentionally less expressive than a general robot program. A small gate should not evaluate arbitrary code. It should check a bounded schema and a bounded set of predicates. More expressive planning belongs upstream; final authorization belongs at the actuation boundary.

4.2 Policy capsule

A policy capsule is a signed, machine-readable object that defines allowed actors, targets, actions, parameter ranges, temporal validity, policy epoch, and required predicates. The capsule may be compiled from a higher-level policy language into a compact representation suitable for microcontroller or FPGA enforcement. A capsule should include:

- policy identifier and hash;
- issuer identity and signature;
- epoch number and rollback-protection metadata;
- allowed action schemas;
- actor-to-capability bindings;
- static bounds such as maximum velocity, force, torque, workspace, and tool state;
- dynamic predicates such as emergency-stop state, guard-zone occupancy, or sensor confidence threshold;
- logging requirements for allow and deny outcomes.

This format intentionally separates high-level policy authoring from low-level enforcement. Human-readable safety policy can live in a version-controlled repository, while Offlyn Verify Core receives a compiled capsule that is smaller, deterministic, and easier to verify.

4.3 Decision semantics

For a request r, policy capsule P_e at epoch e, and trusted state snapshot s, the gate computes:

$$D(r, P_e, s) \in \{\textsc{Allow}, \textsc{Deny}, \textsc{FailClosed}\}.$$

The default decision is FailClosed. The gate returns Allow only if all required checks pass:

1. The policy capsule signature is valid.
2. The capsule epoch is current and not rolled back.

3. The actor is authenticated and authorized for the action type.
4. The request schema is valid and canonicalized.
5. The request is fresh and not a replay.
6. The target actuator or tool is governed by the policy.
7. Static parameter bounds are satisfied.
8. Required state predicates are satisfied.
9. The output command can be encoded without ambiguity.
10. The decision is recorded in the evidence log.

Any failed check yields DENY or FAILCLOSED, depending on whether the gate can produce an authenticated denial record. If logging is unavailable, the gate must prefer fail-closed behavior for safety-relevant commands.

4.4 Minimal enforcement loop

The core enforcement loop can be expressed as follows:

Listing 2: Minimal deterministic enforcement loop.

```
function authorize(request r, state_snapshot s):
    if not valid_policy_signature(P):
        return fail_closed("invalid_policy")
    if rollback_detected(P.epoch):
        return fail_closed("policy_rollback")
    if not fresh(r.nonce, r.timestamp):
        return deny("replay_or_stale_request")
    if not authenticated(r.actor_id):
        return deny("unauthenticated_actor")
    if not schema_valid(r):
        return deny("schema_invalid")
    if not actor_allowed(P, r.actor_id, r.action_type, r.target_id):
        return deny("capability_denied")
    if not static_bounds_hold(P, r):
        return deny("bounds_violation")
    if not dynamic_guards_hold(P, r, s):
        return deny("state_guard_violation")
    cmd = encode_actuator_command(r)
    append_evidence("allow", r, P.hash, s.digest)
    release(cmd)
```

This loop is deliberately simple. The value of hardware rooting increases as the trusted logic shrinks.

5 Architecture

5.1 Components

Offlyn Verify Core consists of six logical components:

1. **Action ingress**: receives canonical requests from the robot executive layer over a deployment-specific transport.

2. **Identity and freshness checker**: validates actor credentials, nonces, timestamps, counters, and policy epoch.
3. **Policy capsule verifier**: checks signatures, policy hashes, validity windows, and rollback metadata.
4. **Deterministic rule engine**: evaluates bounded predicates over request fields and trusted state.
5. **Command release path**: emits the approved actuator command, enables a relay, forwards a bus frame, or releases a cryptographic command token.
6. **Evidence logger**: records allow, deny, and fail-closed decisions with request digest, policy hash, reason code, and monotonic counter.

The key design constraint is that the command release path should be physically or cryptographically unavailable to the AI planner and host software except through Offlyn Verify Core.

5.2 Hardware-rooted implementation options

Several implementation strategies are possible:

- **Inline bus gate**: a microcontroller or FPGA mediates a robot command bus and forwards only authorized frames.
- **Actuator enable gate**: the gate controls an enable line, relay, or drive permission signal while command generation remains upstream.
- **Secure coprocessor**: a protected execution environment authorizes action tokens that downstream drivers require before moving.
- **Safety I/O module**: the gate integrates with existing safety I/O and emergency-stop circuits to provide policy-aware actuation permission.
- **Hybrid gate**: software validators perform rich checks upstream, while hardware enforces a compact set of non-negotiable constraints.

The strongest non-bypass claim comes from inline physical mediation or actuator enable control. A secure coprocessor that merely returns a software decision is weaker unless downstream drivers are designed to require its authorization token.

5.3 Evidence model

For auditability, every decision should produce an evidence record:

Listing 3: Example evidence record.

```
{
  "counter": 88421,
  "decision": "deny",
  "reason": "velocity_limit_exceeded",
  "request_digest": "sha256:...",
  "policy_hash": "sha256:...",
  "policy_epoch": 17,
  "state_digest": "sha256:...",
  "gate_id": "ovc_cell_a_01",
  "firmware_hash": "sha256:...",
  "timestamp": "2026-06-16T18:07:15Z",
  "record_mac": "..."
}
```

Evidence records should be append-only from the perspective of the robot host. A practical implementation may batch records to reduce overhead, but the record counter and message authentication should make missing or reordered records detectable.

6 Relationship to Existing Safety Mechanisms

6.1 Complementarity with LLM robot guardrails

RoboGuard, SafeGate, VerifyLLM-style task verification, and similar systems operate at the semantic and planning level. They can reason about whether a task is unsafe, whether a plan violates a temporal constraint, or whether a natural-language command should be rejected before code generation. Offlyn Verify Core operates at a lower layer. It assumes that upstream systems are helpful but not fully trusted. A command that passed a semantic guardrail must still satisfy physical authorization checks before actuation.

6.2 Complementarity with runtime assurance

Runtime assurance systems typically monitor behavior and switch control to a safety controller when an untrusted controller becomes unsafe. Offlyn Verify Core can be viewed as a narrower enforcement point within a larger runtime assurance architecture. It does not choose an alternative controller; it authorizes or blocks specific actuation requests. In practice, a denied request may trigger a Simplex-style fallback, stop command, or human intervention.

6.3 Complementarity with control barrier functions

CBFs reason about system dynamics and safe sets. They can transform an unsafe nominal control input into a safe input. Offlyn Verify Core should not duplicate high-dimensional control synthesis inside a tiny gate. Instead, a CBF filter can run upstream or in a trusted controller, and Offlyn Verify Core can enforce non-negotiable envelope constraints: maximum speed, force, torque, workspace zone, tool state, freshness, and actor authorization. For richer dynamic safety, the gate can require a signed safety certificate or bounded predicate produced by an upstream verifier, but it should only trust certificates that are fresh and bound to the exact action request.

6.4 Complementarity with mixed-criticality isolation

Mixed-criticality partitioning protects real-time control from interference by perception and application workloads. Offlyn Verify Core addresses a different question: even if the system is temporally isolated, who has final authority to release a safety-relevant action? A static partition or safe I/O cell can be an implementation substrate for Offlyn Verify Core, but the contribution is the action-policy interface and hardware-rooted authorization semantics for AI-originated commands.

7 Evaluation Protocol

A credible evaluation should measure three classes of claims: safety effectiveness, systems overhead, and bypass resistance.

7.1 Safety effectiveness

Construct a benchmark of benign, unsafe, and adversarial action requests across representative robot tasks. For a manipulator, this may include joint movement, Cartesian movement, gripper activation, tool enablement, and restricted-zone navigation. Each task should have a ground-truth expected decision under a fixed policy capsule.

Metrics:

- **Unsafe leakage rate**: fraction of unsafe requests incorrectly allowed.
- **False rejection rate**: fraction of benign requests incorrectly denied.
- **Policy coverage**: fraction of safety-relevant action types represented in the policy schema.
- **Reason-code precision**: fraction of denials whose reason code matches the violated policy.

7.2 Systems overhead

Measure added latency and throughput under realistic command rates. Report median, p95, p99, and worst-case authorization latency. Separate parsing, signature verification, rule evaluation, logging, and command release. For robotics, p99 and worst-case behavior matter more than average latency.

Metrics:

- p50, p95, p99 authorization latency;
- maximum observed latency under load;
- sustained request throughput;
- memory footprint of policy capsules;
- firmware or gate logic size;
- energy or power overhead if measured on embedded hardware.

7.3 Bypass and compromise resistance

Evaluate the system under explicit attack scenarios:

- host attempts to send commands directly to actuators;
- host replays an old allowed command;
- host rolls back to a permissive policy capsule;
- host tampers with a request after upstream approval;
- host suppresses deny logs;
- compromised planner changes actor identity;
- adversarial prompt requests unsafe tool activation;
- sensor predicate is missing or stale.

The primary metric is bypass success rate. A strong implementation should show that host compromise alone cannot release an unauthorized safety-relevant command through the mediated path.

7.4 Comparative baselines

Compare at least four configurations:

1. **No gate**: direct planner or executive access to robot commands.
2. **Software validator**: policy checks run in the same host trust domain as the planner or middleware.
3. **Middleware gate**: ROS-level or service-level pre-execution validation with audit logging.
4. **Hardware-rooted gate**: Offlyn Verify Core mediates the actuation path and logs decisions independently.

The expected result is not that Offlyn Verify Core understands tasks better than semantic guardrails. The expected result is that Offlyn Verify Core reduces bypass success under host compromise and policy rollback while adding bounded latency.

8 Prototype Plan

A minimum viable prototype can be built without fabricating a custom chip. The first implementation should use:

- a robot simulator or low-risk robot platform;
- a scripted or LLM-based planner that emits canonical action requests;
- a software executive layer that translates tasks into movement commands;
- a microcontroller or FPGA development board acting as the gate;
- a signed policy capsule format compiled from a human-readable policy file;
- a mediated command path, such as serial, CAN-like test bus, PWM enable, or simulated actuator driver;
- an append-only evidence log exported for verification.

The first paper can report an emulated or development-board prototype. A later paper can move the deterministic rule engine into FPGA fabric or ASIC layout and evaluate area, timing, power, and fault behavior. The current manuscript should not claim silicon implementation until measured silicon or FPGA results exist.

9 Limitations

Offlyn Verify Core has important limitations:

- **Software prototype**: the current implementation is a software simulation of the proposed architecture. It does not yet include a physical microcontroller, TPM-backed attestation path, FPGA design, or ASIC primitive. The reported latency measurements characterize the prototype software path rather than a hardware enforcement boundary.
- **Structured action scope**: the prototype evaluates structured canonical action requests rather than raw motor-control signals, trajectory streams, or full ROS 2 action graphs. This is intentional for the initial artifact: the goal is to isolate and test the command-release interface, signed policy capsule flow, replay defenses, rollback protection, and fail-closed actuator behavior. Future work should integrate the gate with ROS 2, evaluate physical actuator mediation through a microcontroller, and compare enforcement latency against existing runtime assurance and robot middleware guardrail approaches.
- **Policy correctness**: a hardware gate faithfully enforcing a bad policy can still permit unsafe behavior.
- **State observability**: dynamic predicates are only as reliable as the sensors and trust boundary that supply them.
- **Expressiveness**: a small gate should not run arbitrary planners or high-dimensional optimization. This limits what it can decide locally.
- **Integration complexity**: industrial robots use diverse buses, controllers, safety systems, and certification requirements.
- **Bypass by rewiring**: physical attackers may bypass a gate unless the system is mechanically and electrically designed to prevent it or detect it.
- **Certification**: hardware-rooted policy enforcement is not equivalent to compliance with functional safety standards.
- **Complementarity**: Offlyn Verify Core does not replace formal verification, control-theoretic safety filters, runtime assurance, secure robot middleware, or human supervisory controls. It is intended as a narrow final authorization boundary for AI-generated action proposals before actuator execution.

These limitations are compatible with the paper's thesis. The goal is not to make a robot safe by adding a single box. The goal is to make final AI-originated action authorization smaller, more deterministic, more auditable, and harder for the AI runtime to override.

10 Discussion

The most important design choice is where to draw the boundary. If the boundary is too high, it becomes another software guardrail. If it is too low, it lacks the semantic information needed to make useful policy decisions. The proposed compromise is to require the executive layer to produce canonical, semantically labeled action requests, while the hardware-rooted gate enforces compact policy over those requests and a bounded set of trusted state predicates.

This design also changes how robotic safety evidence can be produced. Instead of asking whether a model "understands" safety, we can ask whether every physical action was released under a specific policy hash, firmware hash, actor identity, state digest, and decision counter. That does not prove global safety, but it turns part of the safety story into reproducible systems evidence.

Finally, Offlyn Verify Core suggests a path toward standardization. Robot vendors and labs could define common action schemas, policy capsule formats, evidence records, and conformance tests. This would let researchers compare actuation-boundary claims across platforms instead of relying on narrative descriptions of guardrails.

11 Conclusion

AI-controlled robots require safety mechanisms that account for both model-level failures and physical consequences. Software guardrails, task validators, runtime assurance, CBF filters, and mixed-criticality isolation are all important, but none alone answers the systems question of where final physical action authority should reside. This paper proposes Offlyn Verify Core, a hardware-rooted actuation policy gate that mediates AI-originated action requests before they reach safety-relevant actuators or tools.

The core claim is intentionally narrow: placing a small, deterministic, signed-policy enforcement point at the actuation boundary can reduce the trusted computing base for robotic action authorization and improve auditability under software compromise. Future work should implement the gate on real embedded hardware, publish a reproducible benchmark suite, measure latency and bypass resistance, and compare against software-only validators in realistic robot deployments.

Artifact Availability

The Offlyn Verify Core v0.1 artifact is publicly available at:

`https://github.com/joelnishanth/offlyn-verify-core/releases/tag/v0.1-paper-artifact`

The artifact includes the paper source, reproducible Python prototype, signed policy capsule flow, deterministic Verify Core gate simulator, actuator simulators, scenario runner, tamper-evident audit-log hash chaining, replay and rollback tests, benchmark scripts, sample benchmark results, and documentation for the staged path from software simulation to TPM, microcontroller, FPGA, and ASIC implementations. All latency measurements reported in this paper are software-simulation measurements and should not be interpreted as hardware performance claims. All tests and scenarios can be reproduced with standard Python tooling and no external services.

References

[1] Z. Ravichandran, A. Robey, V. Kumar, G. J. Pappas, and H. Hassani. Safety Guardrails for LLM-Enabled Robots. *arXiv:2503.07885*, 2025; revised 2026.

[2] I. Obi, V. L. N. Venkatesh, W. Wang, R. Wang, D. Suh, T. I. Amosa, W. Jo, and B.-C. Min. Pre-Execution Safety Gate and Task Safety Contracts for LLM-Controlled Robot Systems. *arXiv:2604.05427*, 2026.

[3] I. S. Cardenas, M. A. Arnett, N. C. Yeo, L. Sah, and J.-H. Kim. ROSClaw: An OpenClaw ROS 2 Framework for Agentic Robot Control and Interaction. *arXiv:2603.26997*, 2026.

[4] T. Ranathunga, K. Fernando, and S. Rea. When Agents Control Robots: A Zero Trust Policy Model for Agentic Cyber-Physical Systems. *arXiv:2605.25653*, 2026.

[5] R. M. Sharma. Ethical Hyper-Velocity (EHV): A Hardware-Rooted Zero-Trust Runtime Enforcement Architecture for Agentic AI Systems. *arXiv:2605.17909*, 2026.

[6] U. Mehmood, S. Sheikhi, S. Bak, S. A. Smolka, and S. D. Stoller. The Black-Box Simplex Architecture for Runtime Assurance of Autonomous CPS. *arXiv:2102.12981*; NASA Formal Methods, 2022.

[7] NASA Technical Reports Server. A Verification Framework for Runtime Assurance of Autonomous UAS. NTRS citation 20240007986, 2024.

[8] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada. Control Barrier Functions: Theory and Applications. *European Control Conference*, 2019.

[9] M. Alshiekh, R. Bloem, R. Ehlers, B. Konighofer, S. Niekum, and U. Topcu. Safe Reinforcement Learning via Shielding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. See also later survey treatments of shielding for safe reinforcement learning.

[10] J. Yen, Z. Huang, Z. Wei, T. Yi, S. Zeng, L. Pang, S. Xue, and Z. Qi. Jiao: Bridging Isolation and Customization in Mixed Criticality Robotics. *arXiv:2605.03641*, 2026.